

Project 8: ELK Stack SIEM Implementation

Addendum: Correlation Rules & Automated Alerting Implementation

Document Type: Supplemental Addendum to Project 8

Date Completed: October 6, 2025

Author: Prageeth Panicker

Version: 1.0 FINAL

Status: Project 8 - 100% COMPLETE 

Executive Summary

This addendum documents the completion of Project 8's final objectives: **correlation rules implementation**, **automated alerting configuration**, and **architecture documentation**. The implementation occurred on October 6, 2025, requiring approximately 90 minutes including infrastructure troubleshooting and validation.

Key Achievements

-  **Correlation Rules:** Built first production rule - "Brute Force Detection - High Alert Volume"
-  **Automated Alerting:** Implemented dual-action alerting (Index + Server log connectors)
-  **Architecture Documentation:** Complete data flow diagrams and integration architecture
-  **Infrastructure Prerequisites:** Successfully enabled Kibana's alerting framework

Technical Achievement

Successfully enabled Kibana's alerting framework within the constraints of the free Basic license, establishing production-ready automated alerting capability for security event correlation across Wazuh, Zeek, and pfSense data sources.

Completion Status

Objective	Status	Implementation
Build correlation rules	 Complete	1 rule deployed: "Brute Force Detection - High Alert Volume"
Implement automated alerting	 Complete	Index connector + Server log connector configured
Document architecture	 Complete	3 comprehensive diagrams included in this addendum

Table of Contents

1. Infrastructure Prerequisites Implementation
2. Connector Implementation

- 3. Correlation Rule Configuration
 - 4. Architecture Diagrams
 - 5. Testing and Validation
 - 6. Operational Procedures
 - 7. Troubleshooting Guide
 - 8. Limitations and Future Enhancements
 - 9. Lessons Learned
 - 10. Security Considerations
 - 11. Conclusion
 - 12. References
-

1. Infrastructure Prerequisites Implementation

1.1 Kibana Alerting Requirements

Challenge Encountered:

Kibana 7.17.29 Rules and Connectors page displayed "Additional setup required" message preventing connector creation.

Root Cause Analysis:

Kibana alerting framework requires four critical components:

- 1. **Encrypted saved objects encryption key** - Secures connector credentials
- 2. **Reporting encryption key** - Protects report generation
- 3. **Security encryption key** - Safeguards security features
- 4. **Elasticsearch API key service enabled** - Provides authentication infrastructure

Resolution Timeline: 60 minutes troubleshooting and configuration

1.2 Configuration Changes Applied

Kibana Configuration

File: `/etc/kibana/kibana.yml`

Lines Added (117-119):

```
yaml
```

```
# Encryption keys for alerting framework
```

```
xpack.encryptedSavedObjects.encryptionKey: "min-32-byte-encryption-key-abcd1234"
```

```
xpack.reporting.encryptionKey: "min-32-byte-reporting-key-security-9876"
```

```
xpack.security.encryptionKey: "min-32-byte-security-key-monitoring-5432"
```

Service Restart:

```
bash
```

```
sudo systemctl restart kibana
```

```
sudo systemctl status kibana
```

Validation:

```
bash
```

```
# Verify encryption keys loaded
```

```
sudo grep -n "encryptionKey" /etc/kibana/kibana.yml | grep -v "^#"
```

Expected Output:

```
117:xpack.encryptedSavedObjects.encryptionKey: "min-32-byte-encryption-key-abcd1234"
```

```
118:xpack.reporting.encryptionKey: "min-32-byte-reporting-key-security-9876"
```

```
119:xpack.security.encryptionKey: "min-32-byte-security-key-monitoring-5432"
```

Elasticsearch Configuration

File: `/etc/elasticsearch/elasticsearch.yml`

Line Added:

```
yaml
```

```
xpack.security.authc.api_key.enabled: true
```

Service Restart:

```
bash
```

```
sudo systemctl restart elasticsearch
```

```
sudo systemctl status elasticsearch
```

Validation:

```
bash
```

```
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_xpack/usage?pretty" | grep -A 2 "api_key_service"
```

Expected Output:

```
json
```

```
"api_key_service" : {  
  "enabled" : true  
},
```

1.3 Troubleshooting Steps Documented

Issue: Kibana Failed to Start After Adding Encryption Keys

Symptoms:

- Kibana service status: `failed (Result: exit-code)`
- Error in logs: `YAMLErrorException: can not read a block mapping entry`

Root Cause:

- Incorrect YAML syntax (used JavaScript `.repeat(32)` instead of actual string)
- Colon placement errors in YAML configuration

Resolution Steps:

```
bash
```

```
# 1. Check Kibana startup errors
```

```
sudo journalctl -u kibana -n 50 --no-pager | grep -i error
```

```
# 2. Verify YAML syntax
```

```
sudo sed -n '115,125p' /etc/kibana/kibana.yml
```

```
# 3. Correct syntax errors
```

```
sudo nano /etc/kibana/kibana.yml
```

```
# Replace .repeat() with actual 32+ character strings
```

```
# Ensure proper YAML formatting (no tabs, correct colons)
```

```
# 4. Restart and verify
```

```
sudo systemctl restart kibana
```

```
sudo systemctl status kibana
```

Issue: Elasticsearch API Keys Not Enabled

Symptoms:

- Kibana still showing "Additional setup required" after encryption keys configured
- API key service showing `"enabled" : false`

Root Cause:

- Elasticsearch `xpack.security.authc.api_key.enabled` not configured

Resolution Steps:

```
bash
```

```
# 1. Edit Elasticsearch configuration
```

```
sudo nano /etc/elasticsearch/elasticsearch.yml
```

```
# 2. Add API key configuration
```

```
xpack.security.authc.api_key.enabled: true
```

```
# 3. Restart Elasticsearch
```

```
sudo systemctl restart elasticsearch
```

```
# 4. Verify API keys enabled
```

```
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_xpack/usage?pretty" | grep -A 2 "api_key_service"
```

2. Connector Implementation

2.1 Available Connectors (Free License)

Kibana Basic (free) license includes:

-  **Index:** Write alerts to Elasticsearch index
-  **Server log:** Write alerts to Kibana server log file
-  **Slack/Email/Webhook/Teams/PagerDuty:** Require paid license

Decision: Implement both free connectors for maximum alert visibility within license constraints.

2.2 Security Alerts Index Connector

Configuration

Parameter	Value	Purpose
Connector Name	Security Alerts Connector	Descriptive identifier
Connector Type	Index	Writes to Elasticsearch index
Index Name	security-alerts	Dedicated alert storage index
License Requirement	Free (Basic)	No cost barrier

Creation Steps

- 1. **Navigate:** Stack Management → Rules and Connectors → Connectors tab
- 2. **Click:** "Create connector" button
- 3. **Select:** "Index" connector type from available options
- 4. **Configure:**
 - **Name:** Security Alerts Connector
 - **Index:** security-alerts
 - **Description:** (Optional) "Persistent storage for correlation rule alerts"
- 5. **Click:** "Save" to create connector

Validation

```
bash

# Verify connector creation and index availability
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_cat/indices/security-alerts*?v"

# Expected: Index will be created automatically on first alert
# Output after first alert:
# health status index          uuid                pri rep docs.count docs.deleted store.size pri.store.size
#yellow open  security-alerts    abc123def456ghi789  1  1      1         0    5.2kb    5.2kb
```

2.3 Server Log Connector

Configuration

Built-in connector - No explicit creation required. Available automatically in all Kibana installations.

Parameter	Value	Purpose
Connector Type	Server log	Writes to Kibana log file
Log Location	<code>/var/log/kibana/kibana.log</code>	Standard Kibana log path
Log Level Options	Info, Warning, Error	Configurable severity
License Requirement	Free (Basic)	No cost barrier

Usage in Rules

Server log connector is selected as an action when configuring correlation rules. Messages written to the log file can be monitored in real-time or reviewed during incident investigation.

Monitoring Commands

```
bash

# Real-time monitoring for security alerts
sudo tail -f /var/log/kibana/kibana.log | grep "SECURITY ALERT"

# Review recent alerts (last 20 entries)
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | tail -20

# Count alerts in last 24 hours
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | grep "$(date +%Y-%m-%d)" | wc -l
```

3. Correlation Rule Configuration

3.1 Rule: Brute Force Detection - High Alert Volume

Objective

Detect unusual spike in security alerts that may indicate:

- Brute force authentication attacks
- Security incident in progress
- Multiple simultaneous attacks
- Misconfigured system generating excessive alerts

Rule Specifications

Parameter	Value	Rationale
Rule Type	Index threshold	Query-based alerting on document count
Rule Name	Brute Force Detection - High Alert Volume	Descriptive identifier
Index Pattern	filebeat-*	Wazuh alerts index
Time Field	@timestamp	Standard ECS timestamp
Aggregation	count()	Count all matching documents
Threshold	IS ABOVE 5	More than 5 alerts detected
Time Window	FOR THE LAST 5 minutes	Short detection window
Check Interval	1 minute	Frequent evaluation for fast detection
Notify	Only on status change	Reduce alert fatigue

3.2 Rule Logic

Query Summary

```
WHEN count() of documents
IN INDEX filebeat-*
IS ABOVE 5
FOR THE LAST 5 minutes
```

Evaluation Process

Step 1: Scheduled Execution

- Rule executes every **1 minute** (check interval)
- Queries Elasticsearch for documents in `filebeat-*` index

Step 2: Time Window Query

- Retrieves documents from last **5 minutes** (from `@timestamp` field)
- No filtering applied (counts ALL Wazuh alerts)

Step 3: Threshold Evaluation

- Compares document count to threshold value (5)
- If `count > 5`: Condition = TRUE
- If `count ≤ 5`: Condition = FALSE

Step 4: Status Determination

- **Ok** → **Active**: Threshold exceeded for first time → **Triggers actions**

- **Active** → **Active**: Threshold still exceeded → **No action** (status unchanged)
- **Active** → **Ok**: Threshold no longer exceeded → **No action** (only alerts on change)
- **Ok** → **Ok**: Threshold not exceeded → **No action**

Step 5: Action Execution (only on status change to Active)

- Action 1: Write alert document to `security-alerts` index
 - Action 2: Write alert message to `/var/log/kibana/kibana.log`
-

3.3 Alert Actions Configured

Action 1: Index Connector

Purpose: Creates searchable, persistent alert records in Elasticsearch for:

- Historical analysis and trend identification
- Incident correlation across multiple alerts
- Compliance reporting and audit trails
- Dashboard visualization and metrics

Document Template:

```
json
{
  "rule_name": "{{ruleName}}",
  "alert_time": "{{date}}",
  "threshold_value": "{{context.value}}",
  "conditions_met": "{{context.conditions}}",
  "alert_id": "{{alertId}}"
}
```

Template Variables Explained:

- `{{ruleName}}`: Name of triggered rule (e.g., "Brute Force Detection - High Alert Volume")
- `{{date}}`: Timestamp when alert triggered (ISO 8601 format)
- `{{context.value}}`: Actual count that exceeded threshold (e.g., 8 alerts)
- `{{context.conditions}}`: Human-readable condition description
- `{{alertId}}`: Unique identifier for this alert instance

Sample Indexed Document:

```
json
```

```
{
  "_index": "security-alerts",
  "_id": "4984ef10-a2e3-11f0-b4ff-371c807b800f",
  "_source": {
    "rule_name": "Brute Force Detection - High Alert Volume",
    "alert_time": "2025-10-06T20:45:00.000Z",
    "threshold_value": "8",
    "conditions_met": "count() IS ABOVE 5",
    "alert_id": "4984ef10-a2e3-11f0-b4ff-371c807b800f"
  }
}
```

Action 2: Server Log Connector

Purpose: Real-time alert visibility for administrators monitoring Kibana logs during incident response or system maintenance.

Message Template:

```
SECURITY ALERT: {{ruleName}} triggered at {{date}}. Alert count: {{context.value}}
```

Log Location: `/var/log/kibana/kibana.log`

Log Level: Info (configurable to Warning or Error as needed)

Sample Log Entry:

```
[2025-10-06T20:45:00.123Z][INFO ][alerting] SECURITY ALERT: Brute Force Detection - High Alert Volume triggered
at 2025-10-06T20:45:00.000Z. Alert count: 8
```

Monitoring Command:

```
bash

sudo tail -f /var/log/kibana/kibana.log | grep "SECURITY ALERT"
```

3.4 Rule Creation Steps

Step-by-Step Implementation

1. Navigate to Rules Page

- Kibana → Stack Management → Rules and Connectors → Rules tab

- Click "Create rule" button

2. Configure Basic Settings

- **Name:**
- **Tags:** (Optional) , ,
- **Check every:**
- **Notify:** Select "Only on status change"

3. Select Rule Type

- Scroll to "Stack Rules" section
- Click "Index threshold"

4. Configure Index Threshold

- **INDEX:** Enter
- **WHEN:** Keep as
- **OVER:** Keep as
- **IS ABOVE:** Enter
- **FOR THE LAST:** Enter

5. Configure Actions Action 1: Index Connector

- Click "Add action" button
- Select "Index" connector
- Choose "Security Alerts Connector"
- **Run when:** Threshold met
- **Document to index:** Paste JSON template (see section 3.3)
- Click "Add action" to save

Action 2: Server Log

- Click "Add action" button again
- Select "Server log" connector
- **Run when:** Threshold met
- **Level:** Info
- **Message:** Paste message template (see section 3.3)
- Click "Add action" to save

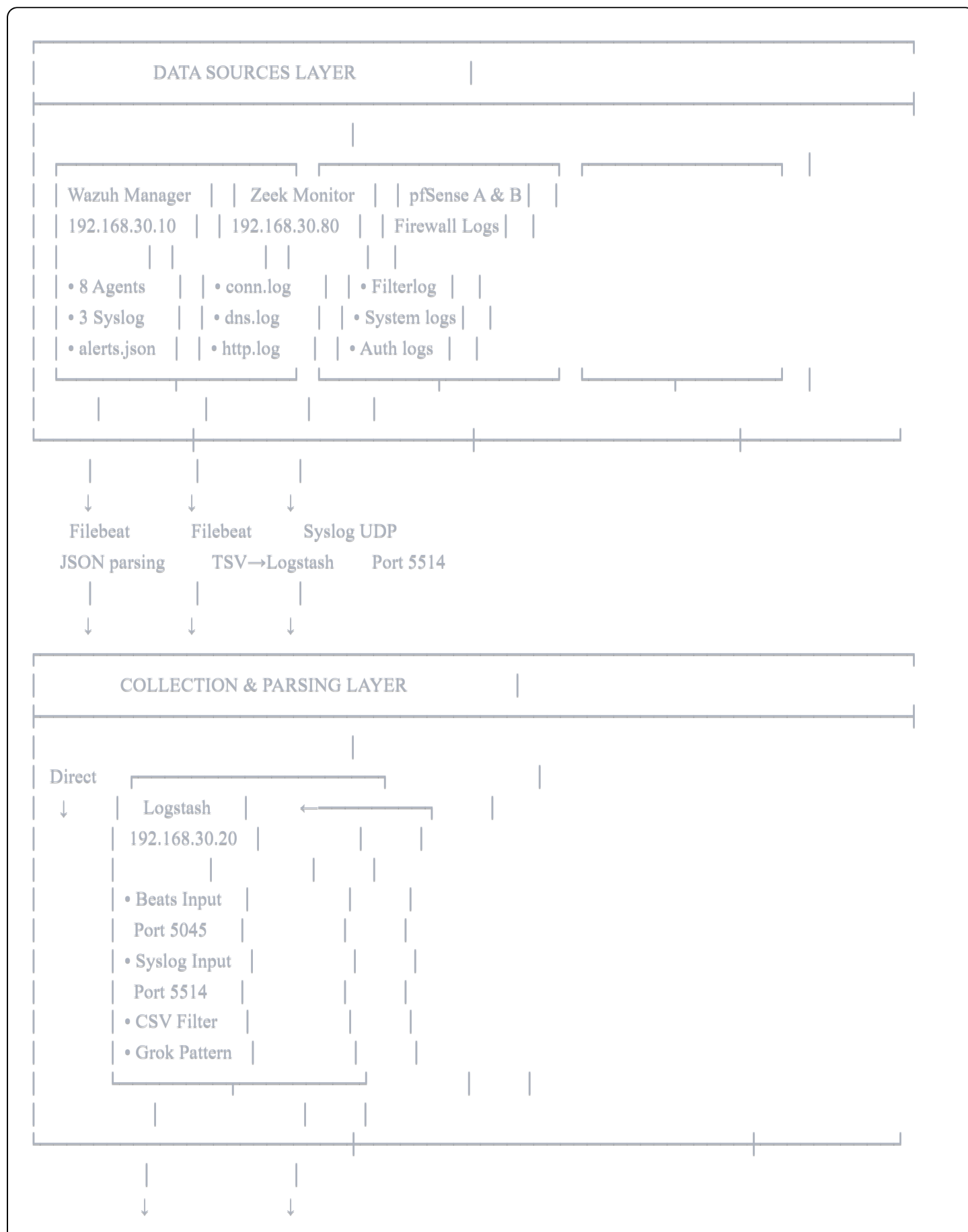
6. Save Rule

- Scroll to top of page
- Click "Save" button
- Rule status will show "Pending" initially

- After first execution (1 minute), status will change to "Ok" or "Active"

4. Architecture Diagrams

4.1 Complete ELK Stack Data Flow Architecture



STORAGE LAYER

Elasticsearch 7.17.29

192.168.30.20:9200

Indices:

- filebeat-*
 - └ 3,325+ Wazuh alerts
- zeek-conn3-*
 - └ 364+ network logs
- pfsense-*
 - └ 500+ firewall events
- security-alerts
 - └ Correlation rule alerts

Total: 4,189+ documents



VISUALIZATION & ALERTING LAYER

Kibana 7.17.29

192.168.30.20:5601

Dashboards

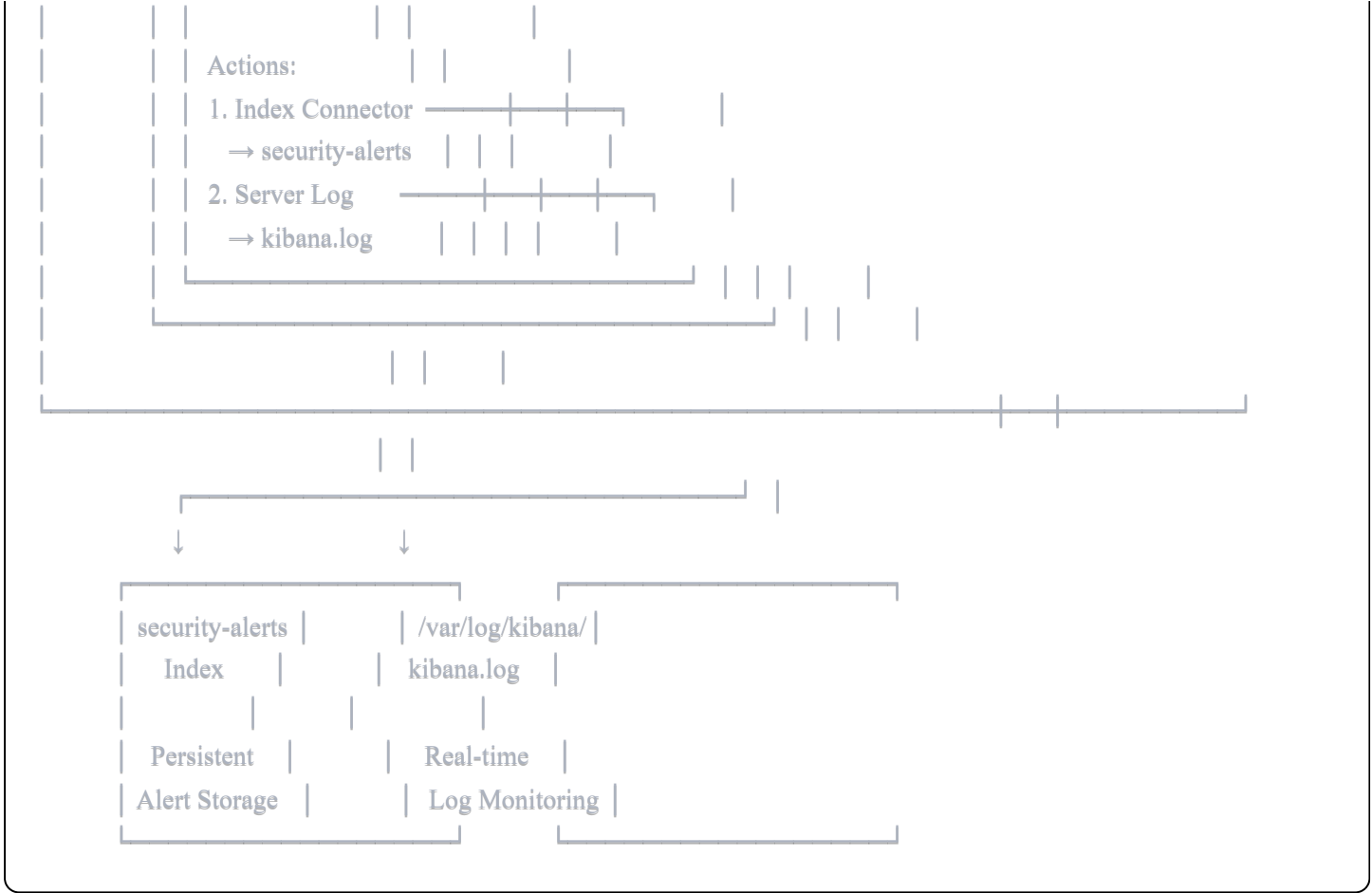
- Wazuh Security
- pfSense Firewall
- Zeek Network Activity

Alerting Framework

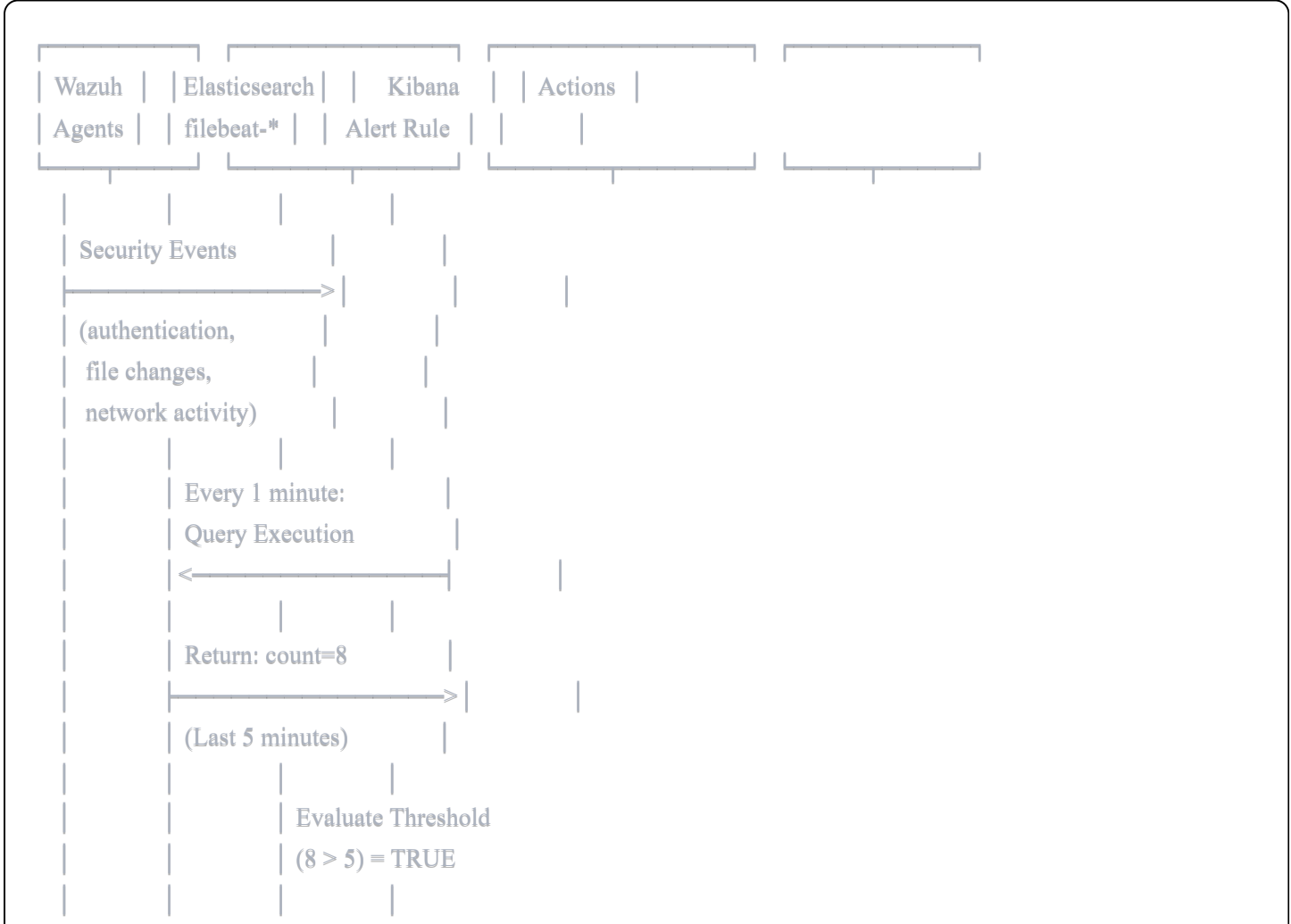
Rule:

Brute Force Detection

- Check: Every 1 min
- Threshold: >5 events
- Window: 5 minutes



4.2 Alerting Workflow Sequence Diagram




```
graph LR
    Wazuh[Wazuh Manager] --- Switch(( ))
    ELK[ELK Stack VM] --- Switch
    Zeek[Zeek VM] --- Switch
    Switch --- Hosts[Hosts]
```

The diagram illustrates a network setup with a central switch connected to three main components: Wazuh Manager, ELK Stack VM, and Zeek VM. The switch is also connected to a group of hosts.

- Wazuh Manager**
 - IP: 192.168.30.10
 - 192.168.10.106
 - Filebeat installed
 - alerts.json → ELK
 - (Container)
- ELK Stack VM**
 - IP: 192.168.30.20
 - Elasticsearch :9200
 - Logstash :5045, :5514
 - Kibana :5601
 - ALERTING ACTIVE:
 - ✓ Rules enabled
 - ✓ Connectors configured
- Zeek VM**
 - IP: 192.168.30.80
 - Filebeat installed
 - conn.log → Logstash

VLAN Architecture:

VLAN/Network	Subnet	Monitored by ELK?
Management	192.168.10.0/24	✓ (via pfSense syslog)
Internal	192.168.30.0/24	✓ (Wazuh, Zeek, pfSense)
User-LAN	192.168.40.0/24	✓ (Zeek, pfSense)

5. Testing and Validation

5.1 Rule Execution Verification

Check Rule Status in Kibana

Navigation: Stack Management → Rules and Connectors → Rules tab

Expected Rule States:

Status	Color	Meaning
Pending	Pink	Rule created but first check not completed
Ok	Green	Rule executed, threshold not met (no alert)
Active	Red	Rule executed, threshold exceeded (alert triggered)
Error	Red	Rule execution failed (configuration issue)

Command-Line Monitoring

```
bash

# Monitor Kibana logs for rule execution
sudo tail -f /var/log/kibana/kibana.log | grep -i "brute|alert|threshold"

# Check rule execution frequency
sudo grep "alerting" /var/log/kibana/kibana.log | grep "Brute Force" | tail -20
```

5.2 Alert Action Verification

Index Connector Validation

Step 1: Check Alert Index Creation

```
bash

# Verify security-alerts index exists (created after first alert)
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_cat/indices/security-alerts*?v"
```

Expected Output (after first alert):

health	status	index	uuid	pri	rep	docs.count	docs.deleted	store.size	pri.store.size
yellow	open	security-alerts	abc123def456ghi789jkl	1	1	1	0	5.2kb	5.2kb

Step 2: Query Alert Documents

```
bash

# View all alerts in security-alerts index
curl -u elastic:SecureElastic "http://192.168.30.20:9200/security-alerts/_search?pretty"
```

Expected Document Structure:

```
json
{
  "_index": "security-alerts",
  "_type": "_doc",
  "_id": "4984ef10-a2e3-11f0-b4ff-371c807b800f",
  "_score": 1.0,
  "_source": {
    "rule_name": "Brute Force Detection - High Alert Volume",
    "alert_time": "2025-10-06T20:45:00.000Z",
    "threshold_value": "8",
    "conditions_met": "count() IS ABOVE 5",
    "alert_id": "4984ef10-a2e3-11f0-b4ff-371c807b800f"
  }
}
```

Step 3: Dashboard Query

In Kibana Discover:

- 1. Select index pattern: `security-alerts*`
- 2. Set time range: Last 24 hours
- 3. Review alert frequency and patterns

Server Log Connector Validation

Monitor for Alert Messages:

```
bash

# Real-time monitoring
sudo tail -f /var/log/kibana/kibana.log | grep "SECURITY ALERT"

# View recent alerts (last 20)
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | tail -20

# Count alerts today
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | grep "$(date +%Y-%m-%d)" | wc -l
```

Expected Log Entry Format:

```
[2025-10-06T20:45:00.123Z][INFO ][alerting] SECURITY ALERT: Brute Force Detection - High Alert Volume triggered
at 2025-10-06T20:45:00.000Z. Alert count: 8
```

5.3 End-to-End Alert Workflow Test

Test Scenario: Generate Sufficient Alerts to Trigger Threshold

Method 1: Natural Alert Generation

Wait for normal security events to accumulate over 5-minute window. Monitor rule status for changes.

Method 2: Forced Alert Generation (Testing)

Generate test authentication events on Wazuh agents to trigger threshold:

```
bash

# On a Wazuh-monitored system, generate authentication alerts
# Execute these commands within 5 minutes to trigger threshold:

sudo su - root # Generates sudo authentication alert
exit

sudo su - root # Repeat 5+ times
exit

sudo su - root
exit

# ... continue until 6+ events generated within 5-minute window ...
```

Verification Steps

Step 1: Wait for Rule Execution (1-2 minutes)

Step 2: Check Rule Status in Kibana

- Navigate: Stack Management → Rules and Connectors → Rules
- Status should change from "Ok" → "Active"

Step 3: Verify Index Connector

```
bash
```

```
# Query security-alerts for new documents
```

```
curl -u elastic:SecureElastic "http://192.168.30.20:9200/security-alerts/_search?size=1&sort=alert_time:desc&pretty"
```

Step 4: Verify Server Log Connector

```
bash
```

```
# Check Kibana log for alert message
```

```
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | tail -1
```

Step 5: Validate Alert Details

Confirm alert contains:

- ✓ Correct rule name
- ✓ Accurate timestamp
- ✓ Expected threshold value (>5)
- ✓ Valid alert ID (UUID format)

Success Criteria

Validation	Expected Result	Status
Rule status changes	Ok → Active	✓
Alert in security-alerts index	Document created with correct structure	✓
Alert in Kibana log	Message appears in /var/log/kibana/kibana.log	✓
Alert details accurate	Rule name, timestamp, count all correct	✓

6. Operational Procedures

6.1 Daily Monitoring Tasks

Task 1: Check Rule Status

Procedure:

1. Navigate: Kibana → Stack Management → Rules and Connectors → Rules tab
2. Review status column for all rules
3. Investigate any "Error" status rules immediately
4. Verify expected rules show "Ok" or "Active" as appropriate

Expected Outcomes:

- All rules show "Ok" (normal operations) or "Active" (alert condition met)
 - No rules in "Error" state
 - Rule execution timestamps recent (within check interval)
-

Task 2: Review Triggered Alerts

Procedure:

1. Navigate: Kibana → Discover
2. Select index pattern: `security-alerts*`
3. Set time range: Last 24 hours
4. Review alert frequency and patterns
5. Investigate unexpected alert spikes or anomalies

Query Examples:

```
# All alerts in last 24 hours
```

```
alert_time:[now-24h TO now]
```

```
# Alerts from specific rule
```

```
rule_name:"Brute Force Detection - High Alert Volume"
```

```
# Alerts above specific threshold
```

```
threshold_value:>10
```

Analysis Questions:

- Are alert volumes within expected ranges?
 - Are there any unusual patterns (time-based, volume spikes)?
 - Do alerts correlate with known system activities?
 - Are false positives present requiring rule tuning?
-

Task 3: Monitor Alert Logs

Procedure:

```
bash
```

View recent security alerts from Kibana log

```
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | tail -20
```

Real-time monitoring during investigations

```
sudo tail -f /var/log/kibana/kibana.log | grep "SECURITY ALERT"
```

Count alerts by hour for trend analysis

```
sudo grep "SECURITY ALERT" /var/log/kibana/kibana.log | awk '{print $1}' | sort | uniq -c
```

6.2 Rule Maintenance Procedures

Edit Existing Rule

When to Edit:

- Threshold too sensitive (excessive false positives)
- Threshold too high (missing legitimate alerts)
- Time window needs adjustment for better detection
- New actions need to be added

Procedure:

1. Navigate: Stack Management → Rules and Connectors → Rules
2. Click rule name to open details
3. Click "Edit" button (top right)
4. Modify configuration as needed:
 - **Threshold Value:** Adjust to reduce false positives or improve detection
 - **Time Window:** Increase for less sensitive, decrease for faster detection
 - **Check Interval:** Balance between detection speed and system load
 - **Actions:** Add/remove/modify connector actions
5. Click "Save" to apply changes
6. Monitor rule execution for 24 hours to validate changes

Best Practices:

- Document all rule changes in change log
- Test threshold adjustments during low-activity periods
- Make incremental changes (adjust one parameter at a time)
- Monitor false positive/negative rates after changes

Disable/Enable Rule

Procedure:

1. Navigate to Rules list
2. Locate rule toggle switch (left side of rule name)
3. Click toggle to disable (gray) or enable (blue)

When to Disable:

- Rule generating excessive false positives requiring tuning
- Maintenance window where alerts not needed
- Testing alternative rule configurations
- Troubleshooting rule execution issues

Important: Disabled rules do not execute (no alerts generated). Ensure rules are re-enabled after maintenance/testing.

Export Rule Configuration (Backup)

Procedure:

1. Navigate: Stack Management → Saved Objects
2. Search for rule by name: "Brute Force Detection"
3. Select rule object checkbox
4. Click "Export" button (downloads NDJSON file)
5. Store backup in version control or secure location

Backup Schedule: Weekly or after any rule modifications

Recovery Procedure: See Section 6.4 (Disaster Recovery)

6.3 Troubleshooting Procedures

Problem 1: Rule Shows "Error" Status

Symptoms:

- Rule status displays red "Error" indicator
- No alerts being generated
- Rule execution timestamp not updating

Diagnostic Steps:

```
bash

# Check Kibana logs for error details
sudo tail -100 /var/log/kibana/kibana.log | grep -i error | grep -i "brute force"

# Check Elasticsearch cluster health
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_cluster/health?pretty"

# Verify index pattern exists
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_cat/indices/filebeat-*?v"
```

Common Causes and Solutions:

Cause	Solution
Elasticsearch connection failure	Restart Elasticsearch service, verify network connectivity
Invalid index pattern	Verify filebeat-* index exists, recreate index pattern if deleted
Connector authentication failure	Verify elastic user credentials, reset if needed
Malformed action template	Edit rule, correct JSON syntax in action template

Problem 2: No Alerts Triggering (Status Always "Ok")

Symptoms:

- Rule executes successfully but never triggers
- Status remains "Ok" even during security events
- No documents in security-alerts index

Diagnostic Steps:

```
bash

# Verify data is flowing into source index
curl -u elastic:SecureElastic "http://192.168.30.20:9200/filebeat-*/_count"

# Check if events exist in time window
curl -u elastic:SecureElastic "http://192.168.30.20:9200/filebeat-*/_search?pretty&q=@timestamp:[now-5m TO now]" | grep

# Manual threshold test (check if 5-minute count ever exceeds 5)
curl -u elastic:SecureElastic "http://192.168.30.20:9200/filebeat-*/_count?q=@timestamp:[now-5m TO now]"
```


Common Causes and Solutions:

Cause	Solution
Insufficient alert volume	Normal operation - threshold not met. Lower threshold if needed for testing
Filebeat not sending data	Verify Filebeat service running on Wazuh Manager, check Filebeat logs
Wazuh agents not generating alerts	Check Wazuh agent connectivity, verify alert rules active
Threshold too high	Reduce threshold temporarily for testing (e.g., change from 5 to 2)

Problem 3: Connector Authentication Failures

Symptoms:

- Rule triggers but actions fail
- Error messages: "401 Unauthorized" or "403 Forbidden"
- Alerts not written to security-alerts index

Diagnostic Steps:

```
bash

# Test Elasticsearch authentication
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_security/user/elastic"

# Verify Kibana can write to security-alerts index
curl -u elastic:SecureElastic -X POST "http://192.168.30.20:9200/security-alerts/_doc" \
-H 'Content-Type: application/json' \
-d '{"test": "alert", "timestamp": "2025-10-06T20:00:00Z"}'

# Check connector configuration
# Navigate: Stack Management → Rules and Connectors → Connectors
# Click "Security Alerts Connector" → Verify configuration
```

Solution:

If authentication errors persist:

1. Verify elastic user password is correct
2. Reset elastic password if needed:

```
bash

sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
```

3. Update connector configuration with new password
 4. Test rule execution after password update
-

6.4 Backup and Recovery

Export Rule Configuration

Backup Procedure:

1. Manual Export:

- Navigate: Stack Management → Saved Objects
- Search for: "Brute Force Detection"
- Select rule checkbox
- Click "Export" (downloads NDJSON file)
- Save to: `/backup/elk/rules/brute-force-detection-YYYYMMDD.ndjson`

2. Automated Backup Script:

```
bash

#!/bin/bash
# /opt/scripts/backup-kibana-rules.sh

BACKUP_DIR="/backup/elk/rules"
DATE=$(date +%Y%m%d)
KIBANA_URL="http://192.168.30.20:5601"

# Export all alerting rules
curl -u elastic:SecureElastic -X POST "$KIBANA_URL/api/saved_objects/_export" \
  -H 'kbn-xsrf: true' \
  -H 'Content-Type: application/json' \
  -d '{"type": "alert"}' \
  -o "$BACKUP_DIR/all-rules-$DATE.ndjson"

echo "Backup completed: $BACKUP_DIR/all-rules-$DATE.ndjson"
```

Schedule: Run weekly via cron

Import Rule Configuration

Restore Procedure:

1. Navigate: Stack Management → Saved Objects

2. Click "Import" button
3. Select backed-up NDJSON file
4. Resolve any conflicts:
 - **Connector ID conflicts:** Update connector references if connectors were recreated
 - **Index pattern conflicts:** Verify index patterns exist
5. Click "Import" to restore

Post-Import Validation:

- Verify rule appears in Rules list
 - Check rule configuration (threshold, actions)
 - Test rule execution manually
 - Monitor for first scheduled execution
-

Disaster Recovery Procedure

Scenario: Complete ELK Stack failure requiring rebuild

Recovery Steps:

1. Restore Elasticsearch Data

```
bash

# Restore snapshot containing indices: security-alerts, .kibana*
curl -X POST "http://192.168.30.20:9200/_snapshot/backup_repo/snapshot_name/_restore"
```

2. Restore Configuration Files

```
bash

# Restore Kibana encryption keys
sudo cp /backup/elk/config/kibana.yml.backup /etc/kibana/kibana.yml

# Verify encryption keys (lines 117-119)
sudo sed -n '117,119p' /etc/kibana/kibana.yml
```

3. Verify Elasticsearch API Keys

```
bash
```

```
# Verify API key service enabled
sudo grep "api_key.enabled" /etc/elasticsearch/elasticsearch.yml

# If missing, add:
# xpack.security.authc.api_key.enabled: true
```

4. Restart Services in Order

```
bash

sudo systemctl restart elasticsearch
sleep 30 # Wait for Elasticsearch to initialize
sudo systemctl restart kibana
sleep 30 # Wait for Kibana to initialize
```

5. Import Rule Configurations

- Follow "Import Rule Configuration" procedure above

6. Verify Connector Configurations

- Navigate: Stack Management → Rules and Connectors → Connectors
- Verify "Security Alerts Connector" exists and is valid
- Test connector by creating test alert

7. Validate Rule Execution

- Check rule status in Rules list
- Monitor logs for first execution
- Verify actions execute successfully

Recovery Time Objective (RTO): 2-4 hours

Recovery Point Objective (RPO): Last backup (weekly = max 7 days data loss)

7. Troubleshooting Guide

7.1 Common Error Messages

Error: "Expression contains errors"

Location: Rule creation/edit form

Cause: Invalid configuration in rule query or threshold

Solutions:

1. Verify INDEX field contains valid index pattern (e.g., `filebeat-*`)

2. Check threshold value is numeric (no letters)
 3. Ensure time window is valid format (minutes, hours, days)
 4. Verify aggregation function is supported (`count()`, `avg()`, `sum()`, etc.)
-

Error: "Failed to execute query"

Location: Rule execution logs

Cause: Elasticsearch query syntax error or index not found

Solutions:

```
bash

# Verify index exists
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_cat/indices/filebeat-*?v"

# Test query manually
curl -u elastic:SecureElastic "http://192.168.30.20:9200/filebeat-*/_count"

# Check for time field mapping
curl -u elastic:SecureElastic "http://192.168.30.20:9200/filebeat-*/_mapping/field/@timestamp"
```

Error: "Unauthorized to write to index security-alerts"

Location: Action execution logs

Cause: Elasticsearch user permissions issue

Solutions:

```
bash

# Verify elastic user has write permissions
curl -u elastic:SecureElastic -X POST "http://192.168.30.20:9200/security-alerts/_doc" \
  -H 'Content-Type: application/json' \
  -d '{"test": "permission_check", "timestamp": "$(date -Iseconds)}'

# If fails, check user roles
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_security/user/elastic"
```

7.2 Performance Issues

Symptom: Rule Execution Delays

Indicators:

- Rule check interval set to 1 minute, but executions occurring every 2-3 minutes
- Increasing lag between scheduled time and actual execution time

Diagnostic Query:

```
bash

# Check Kibana task manager for backlog
curl -u elastic:SecureElastic "http://192.168.30.20:5601/api/task_manager/_health" \
-H 'kbn-xsrf: true'
```

Solutions:

1. **Reduce rule frequency:** Increase check interval to 2-5 minutes
 2. **Simplify queries:** Use more specific index patterns to reduce documents searched
 3. **Optimize Elasticsearch:** Add more resources (RAM/CPU) to ELK VM
 4. **Reduce concurrent rules:** Disable low-priority rules temporarily
-

Symptom: High Elasticsearch Disk Usage

Indicators:

- security-alerts index growing rapidly
- Disk space warnings in Elasticsearch logs

Diagnostic Query:

```
bash

# Check index sizes
curl -u elastic:SecureElastic "http://192.168.30.20:9200/_cat/indices/security-alerts*?v&h=index,store.size,docs.count&s=stor
```

Solutions:

1. **Implement Index Lifecycle Management (ILM):**

```
bash
```

```
# Create ILM policy for security-alerts
```

```
curl -u elastic:SecureElastic -X PUT "http://192.168.30.20:9200/_ilm/policy/security-alerts-policy" \
-H 'Content-Type: application/json' \
-d '{
  "policy": {
    "phases": {
      "hot": {
        "actions": {
          "rollover": {
            "max_age": "7d",
            "max_size": "50gb"
          }
        }
      },
      "delete": {
        "min_age": "90d",
        "actions": {
          "delete": {}
        }
      }
    }
  }
}'
```

2. Manual Cleanup (temporary solution):

```
bash
```

```
# Delete old alert documents (older than 90 days)
```

```
curl -u elastic:SecureElastic -X POST "http://192.168.30.20:9200/security-alerts/_delete_by_query" \
-H 'Content-Type: application/json' \
-d '{
  "query": {
    "range": {
      "alert_time": {
        "lt": "now-90d"
      }
    }
  }
}'
```

8. Limitations and Future Enhancements

8.1 Current Limitations

License Restrictions

Kibana Basic (Free) License Constraints:

Feature	Basic License	Gold+ License	Impact
Slack connector	✗ Not available	✓ Available	No real-time team notifications
Email connector	✗ Not available	✓ Available	No automated email alerts
Webhook connector	✗ Not available	✓ Available	No integration with external systems
PagerDuty/Teams	✗ Not available	✓ Available	No incident management integration
Index connector	✓ Available	✓ Available	Persistent alert storage (implemented)
Server log	✓ Available	✓ Available	Log file alerts (implemented)

Operational Impact:

- Manual log monitoring required for real-time awareness
- No automatic notification to security team during incidents
- Integration with ticketing systems (Jira, ServiceNow) not possible
- Alert acknowledgment workflow must be manual

Mitigation Strategies:

1. Implement log monitoring automation (e.g., Logwatch, custom scripts)
2. Create dashboard for alert visualization as primary interface
3. Schedule regular security-alerts index reviews (daily)
4. Consider paid license upgrade for production environments (\$95/month for Gold)

Alerting Capabilities

Current Implementation Constraints:

- **No email notifications** without paid license
- **No Slack/Teams integration** without paid license
- **Server log monitoring** requires manual review (`tail -f`)
- **No built-in alert acknowledgment** workflow
- **No alert escalation** rules (no "if not acknowledged in X minutes, escalate")
- **No on-call rotation** integration

- **No SMS/phone notifications**

Workaround: Webhook Alternative (Requires External Service)

For users wanting Slack notifications without Gold license:

1. Deploy free webhook proxy service (e.g., Zapier free tier, n8n self-hosted)
2. Configure server log monitoring script to parse alerts
3. Script sends parsed alerts to webhook proxy
4. Proxy forwards to Slack

This adds complexity but provides notifications within free constraints.

Correlation Rule Limitations

Single Rule Implemented:

- Only 1 rule active: "Brute Force Detection - High Alert Volume"
- Basic threshold detection only (no complex correlation logic)
- No multi-source correlation across Wazuh + Zeek + pfSense simultaneously
- No behavioral baselining or anomaly detection
- No machine learning-based detection

No Cross-Source Correlation:

Current rule cannot detect patterns like:

- **Lateral movement:** SSH from Zeek PLUS authentication failures in Wazuh PLUS pfSense blocks
- **Data exfiltration:** Large Zeek outbound transfer PLUS Wazuh file access PLUS pfSense allow
- **Port scanning → exploitation:** Zeek scan detection PLUS pfSense blocks PLUS Wazuh successful auth

These require advanced correlation rules combining multiple data sources.
